GROUP 12 · PRESENTATION PREPARATION · 25 OCTOBER 2026

1. The explanation to memorize first
2. What to study during the remaining hour
3. A 14-minute script aligned with the existing 18 slides
4. Demo: what to click and what to say
5. Complete backend and frontend reference
6. Processing, multimodal AI, and formulas
7. Numbers you can quote, with their exact meaning
8. Evaluator questions and direct answers
9. Deployment position for this presentation
10. Where to look when you need evidence

InfluenceIQ: presentation script, technical explanation, and viva answers

Verified against the current workspace on **25 October 2026**. Read sections 1–4 first. Sections 5–9 are your technical reference and question bank. The older slide deck, current app, historical research studies, and today's software tests have different dates; the distinctions below are intentional.

Companions: complete A–Z guide, existing 18-slide deck, architecture diagram, latest website test evidence.

1. The explanation to memorize first

> InfluenceIQ is a local meeting-review system. It converts recorded speech into a timestamped, speaker-attributed transcript, optionally connects passages to presentation slides, and extracts candidate discussions, actions, decisions, and participant-contribution estimates. Every important candidate retains source references so a person can inspect the original evidence. We evaluate individual components separately instead of inventing one overall accuracy number.

The research title is **InfluenceIQ: A Multimodal Framework for Assessing Participant Contributions to Meeting Decisions**. Meet IQ is another application label in this repository, not a second unrelated project.

An explanation of the name you can use: **Influence** reflects the question of how proposals, evidence, and objections relate to decisions; **IQ** suggests intelligent assistance. This explains why the branding fits the system. It is not a documented historical expansion of the acronym, a personal IQ test, or a validated measurement of someone's intelligence.

Remember these five separations:

Distinction	Explanation
Transcription versus diarization	Words versus anonymous speaker intervals
Discussion versus final decision	Talking about a choice versus committing to one
Source link versus correctness	You can inspect an inference without assuming it is true
Contribution estimate versus causal influence	Observed signals do not prove who caused an outcome
Software tests versus model accuracy	Correct execution does not establish correct interpretation

Your project has three connected parts: the working application, the learned English discussion detector, and exploratory participant-ranking research. Their scores are not interchangeable.

2. What to study during the remaining hour

Use the time you have left rather than trying to memorize every source file. Spend about 8 minutes on the architecture and definitions, 12 minutes rehearsing the script, 12 minutes on the question bank, 8 minutes on the demo, and the final minutes checking the slides, projector, audio, and backup PDF. If time is shorter, prioritize the five distinctions above, the fusion formula, the influence formula, and the result-scope table in section 7.

Keep these ready locally: the existing slide PDF, this script, the architecture PDF, a completed meeting you can legitimately show, and the recorded test-results JSON. Use one prepared result to explain the interface; identify it as prepared or cached when applicable.

3. A 14-minute script aligned with the existing 18 slides

The existing deck is discussion-focused. The newer paper contains additional participant-ranking research. This script explains the app fully while preserving the deck's sequence. The times are rehearsal targets; leave approximately one minute for transitions. These suggested presenter blocks do not establish individual implementation ownership.

Slide 1 — Project identity — 25 seconds

> Good evening. We are Group 12, presenting InfluenceIQ, our multimodal AI meeting-intelligence project. We address the practical difficulty of reviewing a long meeting recording and checking what was discussed, who spoke, and which passages relate to decisions. Our system runs local analysis after model provisioning and preserves the original evidence for human review.

Slide 2 — Introduction — 40 seconds

> A recording preserves a meeting, but finding an important exchange still takes time. A reviewer may need to locate an action, check a proposed decision, or understand who supplied the evidence behind a discussion. Our application connects those review tasks to timestamped transcript passages. Its purpose is to make the meeting easier to inspect, with the recording available whenever an automatic interpretation needs checking.

Slide 3 — Problem statement — 45 seconds

> Consider three statements: “We should launch on Friday,” “Testing needs another two days,” and “We have decided on Monday.” They express a proposal, a constraint, and a decision. The person announcing Monday may not be the person whose evidence changed the choice. We therefore distinguish discussion detection, candidate decision reconstruction, and participant contribution. A fluent-looking result or a large speaking-time total cannot establish all three.

Slide 4 — Objectives — 40 seconds

> Our objectives are local recording and upload, speaker-attributed transcripts, optional presentation alignment, source-linked candidates, and editable review with history and exports. We also compare models on defined research tasks. The system supports English/Hindi speech settings and bilingual event rules, while the installed learned discussion detector is English-only. Those language capabilities have different evaluation boundaries.

Slide 5 — Project plan — 30 seconds

> The work progressed through scope definition, dataset preparation, component comparison, pipeline integration, interface development, and validation. We built the application around reusable stages and preserved experimental artifacts. The latest website regression passed 656 tests, although the earlier deck retains a historical testing snapshot. Independent final validation of contribution-to-outcome judgments remains further research.

Slide 6 — Literature review — 40 seconds

> Existing work supplies strong foundations in speech recognition, speaker diarization, meeting assistants, decision discussion, and evidence-grounded summarization. We use established components where appropriate. Our contribution is the concrete integration, source-preserving review representation, and exploratory evaluation of participant-contribution features. We compare methods on the same task and inputs rather than placing unrelated published scores beside our own.

Slide 7 — Research and engineering gap — 40 seconds

> Our engineering challenge is to connect speech, speakers, optional slide content, and structured analysis into a usable local workflow. Our research question asks whether contribution-related features provide information beyond basic participation measures. Traceability helps expose failures, but it does not prove that an inferred relation is correct. We do not claim to invent meeting influence analysis or to have established causal attribution.

Slide 8 — Proposed solution / architecture — 100 seconds

> Read the architecture in three passes. First, the React interface records or accepts audio and optional PowerPoint files. FastAPI authenticates the user, validates the inputs, saves them, and queues one analysis job. FFmpeg converts ordered audio parts into a compatible recording.
>
> Second, Whisper supplies words and timestamps. Sherpa-ONNX estimates anonymous speaker intervals, and overlap-based attribution assigns words to those intervals. Draft text appears before global speaker analysis finishes. Recognition observations are cached separately, so correcting speakers can reuse the original words. Optional slide text is extracted and compared with passages.
>
> Third, two analysis branches remain separate. A trained English model suggests decision-related discussion passages. Event rules, evidence relations, candidate decision clusters, and interactions support structured outputs and weighted contribution estimates. Reports and SQLite history feed the review interface, where people can listen, correct labels, review tasks, and export results. These arrows describe dependencies, not simultaneous execution.

Slide 9 — Requirements — 40 seconds

> Functional requirements cover recording, ordered multi-file uploads, optional presentations, transcript review, participant edits, and exports. Reliability requirements include input validation, visible progress, reconnecting, saved jobs, and reusable completed chunks. The browser keeps recording chunks in IndexedDB. The backend retains audio and stage artifacts. Recovery preserves completed work where possible; it does not guarantee that an interrupted in-progress model call has completed.

Slide 10 — Technology stack — 55 seconds

> The browser uses React, Vite, Axios, custom CSS with Tailwind, and native recording APIs. FastAPI and Python coordinate the backend. FFmpeg standardizes audio; faster-whisper runs pretrained Whisper recognition; Sherpa-ONNX runs pretrained speaker models. Python-pptx reads presentation content, and MiniLM embeddings or TF-IDF provide text similarity. Scikit-learn supplies the discussion classifiers and research baselines. SQLAlchemy and SQLite manage accounts and history, while local files hold recordings and intermediate artifacts. Native Android is a separate Java/C++ implementation with whisper.cpp.

Slide 11 — Discussion detector — 65 seconds

> The learned task is discussion membership. Training examples are AMI reference dialogue acts, labeled positive when at least half their lexical word identifiers fall inside annotated decision-discussion spans. We combine current-act word features, character features, and context from up to forty acts before and after. The installed score is 75 percent random forest and 25 percent logistic regression, followed by five-neighbor smoothing on each side and a development-selected threshold. Future context makes it an offline model. It suggests passages; it does not directly decide which proposals were adopted.

Slide 12 — Implementation and fusion — 65 seconds

> The input sources are audio and presentation documents. Audio reaches slide alignment through transcript text; slides contribute extracted text, tables, and notes. We calculate semantic cosine similarity and, when display times are supplied, temporal overlap. Their weighted average is score-level fusion, often called late fusion. Defaults give each component weight 0.5, keep three candidates, and abstain below 0.3. Without slide timing, the default becomes semantic-only. TF-IDF is an alternative content scorer, not an extra model averaged with MiniLM. The implementation does not contain a trained acoustic-image cross-attention network.

Slide 13 — Testing and debugging — 55 seconds

> Today’s regression passed 656 Python tests, and frontend lint/build and browser checks passed. The browser tests cover authentication errors, recording recovery, multiple uploads, progress, reconnection, complete draft history, automatic result updates, and exports. A real 90-second English example with two audio parts and two presentations completed with four speaker groups. We also corrected a cutoff that suppressed recognized proposal and evidence cues. These checks validate execution and integration. They do not turn software-test counts into speech or influence accuracy.

Slide 14 — Discussion results — 65 seconds

> The installed discussion model achieved 53.54 percent historical F1, with 39.94 percent precision and 81.18 percent recall. This means it retrieves many reference discussion acts but also produces many false positives. The matched context forest alone scored 52.08 percent F1, so the observed difference is 1.46 percentage points. The inputs were human transcripts and dialogue-act boundaries, and the historical meetings represent only three independent project series. This is an exploratory component result, not complete recording-to-decision accuracy.

Slide 15 — Speech, speakers, and runtime — 65 seconds

> We measure speech errors separately. On the small Hindi pilot, Turbo reduced word error rate from 50.96 to 28.85 percent; Hinglish changed from 66.13 to 57.26 percent. Historical English worsened, so small remains the shared default. A separate small speaker-calibration pilot reduced diarization error from 64.08 to 40.91 percent. These are limited component studies. In today’s functional run, the 90-second input took about 41.84 seconds of recorded processing, mostly transcription and speaker analysis. An unchanged cached rerun took 0.60 seconds, which is a different timing condition.

Slide 16 — Conclusion — 40 seconds

> InfluenceIQ connects local meeting processing with inspectable review. Its strengths are source preservation, an integrated web/mobile workflow, explicit stage status, and transparent comparison of component methods. The current evidence does not establish universally accurate decision extraction, causal influence, or a guaranteed benefit from adding slides. Our next research priority is independently annotated, fresh recording-level evaluation, including contribution-to-outcome relationships and matched slide/no-slide conditions.

Slide 17 — References — 10 seconds

> The references support the speech, speaker, meeting-assistant, dataset, and discussion-analysis components. The full bibliography and experimental protocols are retained in the project records. Thank you; we welcome your questions.

Slide 18 — Photograph — no spoken time

This is a post-presentation photograph placeholder. For questions, return to architecture, discussion results, or conclusion.

If asked about the newer paper: “The latest controlled participant-ranking study reports 68.93 percent meeting-average pairwise accuracy against GAP outcome alignment. That is a separate exploratory task. The paired uncertainty interval includes zero, so we do not claim established superiority.”

4. Demo: what to click and what to say

The website is available at **http://127.0.0.1:8001**. Port 5174 is the development frontend. The built website at 8001 is sufficient for the demo. Use your own account and a completed recording you have permission to show.

Screen / action	What to explain
Login / register	Local account identity and private owned history. Passwords are hashed. There is no email-based recovery service.
Home	Record or upload audio; add and reorder multiple parts and PPTX decks. Parts join chronologically into one meeting.
Language / expected speakers / vocabulary	These are hints and constraints, not correctness guarantees. Leave speaker count unknown rather than guessing.
Processing	Actual stages, elapsed time, measured audio counters, and estimated completion. Drafts retain available passages from the start.
Overview	Duration, speaker groups, candidate counts, warnings, vocabulary, and timing. A speaker group is not a verified identity.
Transcript	Search, speaker filtering, pagination, and timestamps. Play one passage to demonstrate traceability.
Participants	Consecutive display labels plus editable names/roles. Naming a cluster does not repair a wrong acoustic assignment.
Action items	Task candidates retain source evidence. A reviewer explicitly sets owner, exact date, and status.
Decisions	Distinguish learned discussion suggestions from rule-reconstructed decision candidates. Open source/lineage details.
Influence	Show raw features, normalized features, weights, and source events. Explain zero scores and the absence of causal validation.
Events	Show the event type, source text, speaker, and recording time.
Presentation	Show extracted slide content and linked passages. A related slide is not proof that a spoken statement is true.

Screen / action	What to explain
Export	JSON, Markdown, text, subtitles, CSV, and reviewed-task calendar; browser print supplies PDF. Calendar export is not live Google/Outlook sync.
History / settings	Saved meetings, account profile, language/model defaults, and speaker setup.

A concise demo sentence: “I will open one candidate, inspect its source words, play the timestamp, and show how a human correction is saved.”

If a result is empty, explain the observed state: “No candidate passed this stage's rules or threshold.” Do not substitute a fabricated output. If the backend cannot be reached, use the local slide PDF and saved report/screenshot as a clearly identified backup.

5. Complete backend and frontend reference

5.1 Technology and responsibility

Technology	Responsibility	Why it fits this implementation
Python 3.11	Processing and orchestration	Existing speech/NLP/data libraries and shared CLI/API modules
FastAPI + Uvicorn	HTTP application and server	Typed validation, structured APIs, file handling, integration with Python processing
Pydantic	Validate request fields	Bounds, allowed modes, finite/structured values at interfaces
SQLAlchemy + SQLite	Accounts, owned meetings, serialized reports, sync links	Durable local transactions without another database service
bcrypt	Password hashing	Passwords are checked against hashes rather than stored as plaintext
PyJWT	Expiring access and scoped media tokens	Authenticated account access and bounded recording playback access
FFmpeg / FFprobe	Decode and inspect media	Common recording formats and compatible model input
faster-whisper / CTranslate2	Local Whisper inference	CPU/int8 execution with timestamped output
Sherpa-ONNX	Local speaker inference	Segmentation, embeddings, and whole-meeting clustering
python-pptx	PPTX extraction	Native text, tables, and notes
Sentence-transformers / MiniLM	Sentence vectors	Semantic transcript–slide similarity
scikit-learn / SciPy	Discussion classification, sparse matrices, research models	Reproducible conventional supervised methods
NumPy / SoundFile	Waveform and numerical operations	Audio samples, metadata, and intermediate matrices
React 19 / React DOM	Browser interface	State-driven capture, progress, review, and edits
Vite	Development and production frontend build	Development proxy and compiled static assets
Axios	HTTP calls and upload progress	JWT headers, API errors, polling, downloads

Technology	Responsibility	Why it fits this implementation
Tailwind + custom CSS / Lucide	Styling and icons	Responsive controls and consistent visual hierarchy
MediaRecorder / Web Audio / AudioWorklet	Browser capture, level display, preview windows	Platform recording and audio processing features
IndexedDB	Browser recording chunks	Recovery after reload/interruption where chunks were saved
pytest / Playwright / oxlint	Validation	Python checks, browser flows, frontend lint

An installed dependency is not necessarily an active feature. For example, the current interface's contribution meters do not establish that a graph neural network, PageRank, or a general analytics-chart engine is being used.

5.2 Backend request lifecycle

`backend/api.py` owns the HTTP boundary. A web upload includes title, model/language choices, vocabulary, diarization mode, optional speaker count, ordered audio files, and ordered PPTX files. The API permits 1–32 audio parts and at most 32 decks, rejects empty/invalid files, and defaults to a configurable aggregate limit of 1 GiB.

It generates a UUID-based meeting ID, retains uploads, saves durable job JSON, inserts a SQLite meeting row, and returns HTTP **202 Accepted**. A `ThreadPoolExecutor(max_workers=1)` processes jobs one at a time. This is a local serialized worker, not a distributed Celery/Redis queue. A file lock prevents another backend using the same worker runtime.

The background runner decodes ordered audio parts, invokes `run_meeting()`, copies the completed report into the meeting row, preserves applicable action-review history, and marks completion. Exceptions record a failed state and retain source audio. On startup, saved queued/processing jobs with durable state can be resubmitted.

The three database tables are `users`, `meetings`, and `meeting_sync_links`. A meeting stores its owning user ID, title, status, serialized report, and creation time. Waveforms and model checkpoints remain files. Accounts use bcrypt hashes and expiring JWT access; meeting routes enforce ownership. Audio playback uses a separate scoped token. This is application access control, not encrypted-at-rest audio storage or public-hosting certification.

5.3 API map

API	Purpose
`POST /api/auth/register`	Create a local account when registration is enabled
`POST /api/auth/login`	Check credentials and issue an access token
`GET /api/auth/me`	Account identity

API	Purpose
`GET /api/health`	Local model/setup readiness, not measured accuracy
`POST /api/recording/preview`	Bounded draft recognition windows; may return busy status
`POST /api/analyze`	Validate uploads and queue full processing
`GET /api/meetings`	Owned meeting history
`GET /api/meetings/{id}`	Status, progress, preview, saved report, revision
`GET /api/meetings/{id}/audio`	Authorized recording playback
`POST /api/meetings/{id}/retry`	Retry a failed job from retained sources/state
`POST /api/meetings/{id}/analyze-speakers`	Explicit speaker reanalysis with count/revision checks
`POST /api/meetings/{id}/edit`	Supported title/speaker/vocabulary edits
`POST /api/meetings/{id}/actions/{event_id}`	Reviewed owner, date, and task state
`GET /api/meetings/{id}/export`	Download a supported format
`GET/POST /api/profile`	Saved account profile
`POST /api/sync/meetings`	Explicit device report/optional audio publication
`GET /api/sync/meetings/{id}/audio`	Owned synchronized audio download

Password-reset HTTP routes are intentionally disabled; the project has a local administrator CLI. Do not demonstrate them as an email integration.

5.4 Frontend lifecycle

`App.jsx` holds authentication, recording, upload, selected-meeting, result-tab, speaker-edit, and preference state. `useState` controls visible data; `useEffect` handles lifecycle/fetching/polling; refs retain recording resources and request generations. Axios adds the access token to API requests. The browser never queries SQLite directly.

`recordingStore.js` saves browser chunks in IndexedDB. `livePreview.js` uses a 16 kHz AudioWorklet and sends approximately 15-second preview windows to the local backend. These drafts are not final diarization. The full saved recording is used for final analysis, including windows skipped when the preview worker is busy.

`ProcessingPanel.jsx` presents actual stage information, counters, drafts, and explicitly estimated completion. Polling is approximately every 1.5 seconds while work/results are pending; this implementation does not use a WebSocket stream. Request-generation and selected-meeting checks prevent stale responses from replacing the wrong meeting. Incomplete completion responses and initial connection failures keep retrying.

`TaskReview.jsx` collects human follow-up corrections. Reviews bind to the event and source text/time; revision checks prevent stale saves. An extracted task's speaker is not silently assigned as its owner, and relative dates are not silently converted into exact calendar dates.

The production browser loads `frontend/dist`; the backend serves those static assets. Node/Vite is needed for development/build, not as a separate production frontend server. Development port 5174 proxies API calls to backend port 8001.

5.5 Storage map

Location	Content
`config/config.yaml`	Processing defaults and prototype thresholds/weights
`data/state/nexus_v2.db`	Main local account/history database
`.runtime/jwt-secret`	Persistent local token-signing secret
`.runtime/uploads/`, `.runtime/jobs/`	Inputs and durable job state
`data/custom_test/`	Default retained decoded API recordings
`data/processed/audio/`	Transcript, speaker, word-observation and chunk artifacts
`data/processed/slides/`, `alignment/`	Extracted decks, embeddings, alignment
`data/processed/events/`, `decisions/`, `interactions/`, `influence/`	Source-linked analytical structures
`outputs/reports/`	Report JSON/Markdown/HTML/CSV files
`professor_guidelines/evaluation/`	Component study metrics and protocols
`docs/paper_20261024/analysis/`	Latest controlled ranking study
`outputs/evaluation/website_regression_20261025.json`	Today's website checks and timings

6. Processing, multimodal AI, and formulas

6.1 Audio and attribution

Normalize to **16 kHz, mono, PCM16**. This means 16,000 two-byte samples each second, approximately 115.2 MB/hour uncompressed. The conversion standardizes input; it does not automatically denoise or repair clipping.

Current laptop defaults are Whisper small, CPU/int8, four threads, beam size five, and 120-second central chunks. Approximately two seconds of acoustic context is read around chunk boundaries. Word-midpoint ownership assigns each word to one central interval. The full orchestrator uses faster-whisper's VAD filtering; standalone Silero VAD is also available to the separate stage runner.

Diarization uses pretrained segmentation and speaker embeddings through Sherpa-ONNX, followed by whole-meeting clustering. The configured threshold is 0.9. This is distinct from the alternative full Python pyannote backend. Whole-meeting speaker processing is not bounded-memory streaming and does not acoustically separate simultaneous voices.

For a word `[a,b]` and speaker interval `[c,d]`:

```
overlap = max(0, min(b,d) − max(a,c))
coverage = unioned same-speaker overlap / word duration
```

Select the greatest qualifying coverage, with default minimum 0.3. Insufficient coverage, exact ties, or an insufficient configured margin give UNKNOWN. Anonymous stored IDs remain stable source references; the interface displays consecutive Speaker 1, Speaker 2 labels in first-appearance order.

Draft recognition precedes global speakers; final attribution reuses separately cached word observations. Diarization `auto` can continue with UNKNOWN on failure; `required` fails; `off` deliberately omits speakers. Participant influence is unavailable without valid speaker separation.

6.2 Modalities and fusion

There are two input sources: **audio** and **presentation documents**. In the deployed slide branch, audio is represented as recognized text and timing, and PPTX contributes extracted text, tables, and notes. Without PPTX, the application follows its audio-only path. Text, speaker labels, and timestamps derived from one recording are not three independent captured modalities.

Default semantic matching uses `all-MiniLM-L6-v2` sentence vectors and cosine similarity. TF-IDF supplies an alternative lexical content score. Optional local Tesseract OCR can read embedded image text when enabled; current OCR is disabled. OCR does not create general image/diagram understanding.

```
S = cosine(passage vector, slide vector)
T = passage/slide display-time overlap / passage duration
```

```
C = (wt × T + ws × S) / (wt + ws)
```

Defaults: `wt=0.5`, `ws=0.5`, `top_k=3`, threshold `0.3`. Missing timing stays unavailable; with the default nonzero content weight, use semantic-only scoring. Reject weak matches with `NO_CONFIDENT_SLIDE`.

This is **weighted score-level fusion**, commonly categorized as late fusion. TF-IDF replaces MiniLM for that run; the two content methods are not jointly averaged. Speaker attribution is another temporal integration operation, and forest/logistic combination is a model ensemble. Neither should be confused with a trained audio-image fusion network.

The joined `MultimodalSegment` carries source ID, speaker, text, timestamps, slide ID, and component/combined scores into downstream event analysis. There is no deployed cross-attention, tensor/bilinear fusion, learned multimodal gate, visual encoder, or end-to-end multimodal training objective.

6.3 Events, evidence, and decisions

The default event stage uses English/Hindi/Hinglish patterns and blockers, with one winning role per passage: proposal, question, objection, evidence, correction, clarification, agreement, disagreement, revision, decision, or action item. Nonmatching passages remain unclassified.

The original match-density ranking still selects the role. Today's scoring version 2 prevents a recognized English cue being rejected just because its role has more alternative patterns: confidence includes a `0.3 × English_hits` floor, capped at one. The default threshold is 0.3. These are heuristic scores, not correctness probabilities. The updated rule behavior has functional tests, not a new independent NLP accuracy benchmark.

Evidence relations use role/proximity rules, normally within 60 seconds. Candidate decisions collect events around explicit decision or qualifying agreement anchors, normally within 120 seconds. With no resolved anchor, default non-strict mode can retain proposal candidates. Statuses are confirmed, probable, conflicting, or unresolved; “confirmed” means an explicit decision-type event was detected, not that a human adjudicated the outcome.

Interactions combine evidence edges, decision lineage, and reactive-event associations. The graph stores inferred relationships. Temporal order and graph arrows do not establish causality.

6.4 Default influence calculation

Feature	Weight	Abbreviated raw observation
Proposal	0.15	Observed proposals in decision clusters
Evidence	0.20	Linked evidence, with a slide-grounded bonus
Objection	0.15	Linked challenges, with a later-revision association bonus

Feature	Weight	Abbreviated raw observation
Revision	0.20	Revisions in probable/confirmed clusters
Decision	0.15	Participant events in probable/confirmed clusters
Interaction	0.05	Outgoing decision-linked interactions
Slide grounding	0.10	Linked events associated with a slide

```
z(p,k) = x(p,k) / max_q x(q,k), or zero if the category is inactive
score(p) = Σ_k weight(k) × z(p,k)
```

Known transcript speakers are retained even when their features are zero. UNKNOWN is excluded. Equal scores receive shared ranks. Features can overlap, weights are prototype choices, and missing categories stay zero. The remaining weights are not automatically rescaled merely because slides or proposals are missing.

Consequently, a top score may be 0.15 or 0.70 instead of 1.00. Scores do not sum to one across people. A zero means no scored decision-linked contribution was extracted; it may reflect missing events or wrong speakers. The maximum-normalized weighted score is not PageRank, a probability, or a personal performance rating. Experimental learned rankers are separate artifacts, not the default UI scorer.

6.5 Learned discussion detector

The deployed English detector uses train-fitted word/character TF-IDF plus up to ±40 neighboring input units, within meeting boundaries. Word features include unigrams/bigrams; character features use within-word 3–5 grams. The installed random forest/logistic score mixture is 0.75/0.25, smoothed over up to ±5 units, threshold **0.3577398635496262**.

Research labels use AMI manual decision-discussion spans and reference dialogue acts. Splits contain 29 training meetings, six development, and twelve historical meetings. All meetings from a project series remain together. Training fits parameters; development selects settings; repeated inspection makes the historical cohort exploratory. The deployed model receives ASR passages rather than oracle acts, so its historical F1 does not establish application accuracy.

6.6 Android and optional research branches

Android is a separate native Java/C++ app with whisper.cpp, native Sherpa-ONNX, local recording/checkpoints, native-text TF-IDF, and simpler candidate lineage. Local analysis needs no account after model setup. Shared account publication/download is explicit; sign-in does not upload recordings. The current documented prototype has INTERNET permission for optional sync. Do not claim exact Python model/output parity or physical-phone performance from emulator checks.

Research modules also support contextual role/discussion models, semantic topic grouping, outcome-pair candidates, and XGBoost ranking. They are optional experiments. The AI outcome-link pilot used

assistant labels, which do not provide independent human ground truth; it did not beat its majority baseline.

7. Numbers you can quote, with their exact meaning

Number	What it measures	What to say with it
656 passed	Python software tests, latest website regression	Functional checks, not 656 correct meetings
41.84 s	Recorded processing for 90-second English speech, two parts/two decks/four-speaker constraint	Warm models, uncached meeting inputs; excludes upload/queue/decode/final persistence
0.60 s	Unchanged cached reanalysis of that input	A separate cache condition, not fresh inference speed
53.54% F1	Installed English discussion detector on historical AMI reference acts	Human text/boundaries, exploratory historical cohort
39.94% precision / 81.18% recall	Same discussion detector	Many reference discussions retrieved, many false positives
68.62% accuracy	Same discussion detector	Imbalance makes accuracy insufficient by itself
77.72% accuracy	Always-negative discussion baseline	Finds zero positive discussion despite high accuracy
68.93% pairwise accuracy	Latest combined Ridge ranking against GAP outcome alignment	Different task, 28 previously examined meetings; not causal or whole-system accuracy
+7.14 points	Combined minus participation-only Ridge	Descriptive paired interval −1.79 to +17.86 crosses zero
61.11% versus 63.89%	Original six-meeting GAP selected Extra Trees versus word count	Selected model did not beat that baseline; different protocol from latest 28-meeting study
50.96% → 28.85% WER	Small versus Turbo on eight Hindi samples	Tiny speech pilot; lower is better
66.13% → 57.26% WER	Small versus Turbo on eight Hinglish samples	Tiny speech pilot; not reliable multilingual meeting performance
61.61% → 63.91% WER	Historical English small versus Turbo	Regression retained, not hidden; small remains shared default
64.08% → 40.91% DER	Separate two-excerpt speaker calibration pilot	Custom small pilot, reference/protocol limitations apply

Fusion results, particularly relevant to your open files: on 24 author-designed cases, temporal-only top-1 was **50%**, semantic-only **83.33%**, and the fixed hybrid **66.67%**. Semantic-only performed best on

that constructed set. Equal temporal weight can favor an unrelated visible slide or an offscreen-reference error. Do not claim the hybrid universally improved accuracy.

The real unlabelled comparison has **1,008 passages, 34 slides, zero slides with valid display times**. Semantic and hybrid selected the same links for 39 passages, coverage **3.87%**. Their agreement is 100%, but accuracy/F1 are unavailable because independent relevance labels are missing. Agreement and coverage are not correctness.

The latest GAP controlled comparison uses 28 meetings, 84 people, 8,009 reference utterances, and 87 reference-comparable participant pairs. Seven outer group folds hold out four meetings each; three inner folds select Ridge alpha from 0.1, 1, 10, and 100. The compared representations have six participation, seven contribution, or thirteen combined features. Word count reaches 66.55%, duration 67.86%, contribution-only Ridge 53.39%, and fixed contribution score 55.77% under that protocol.

Dataset roles: AMI supplies reference meeting audio/transcripts and the custom 47-meeting discussion split. GAP supplies the participant outcome-alignment study. The eight-sample Hindi speech pilot uses Google FLEURS `hi_in` test audio; the eight Hinglish samples use a MUCS-Hinglish mirror referring to OpenSLR SLR104 and are consecutive lecture excerpts, not independent meetings. Their pinned manifests are `professor_guidelines/evaluation/hindi_manifest.json` and `hinglish_manifest.json`. Generated slide cases and today's simple test decks are integration/controlled fixtures, not independently annotated real multimodal meetings.

Historical coverage diagnostics found 70 of 84 participants without contribution features, and no emitted proposals among 537 reference-tagged proposals in that evaluated snapshot. These diagnostics predate today's confidence correction and were not rerun as an accuracy benchmark. A better intermediate role metric does not automatically validate personal influence.

Primary files: discussion scorecard, fusion controlled cases, real unlabelled alignment, latest ranking results, website evidence.

8. Evaluator questions and direct answers

Answer the question first, then give the mechanism, a concrete example or measurement, and the boundary if it matters. Explain the boundary once clearly; do not repeat every limitation after every sentence.

Purpose and contribution

1. What exactly did you build? A local audio-to-review application, source-linked structured analysis, native Android support, and scoped component/research comparisons. We integrated pretrained speech/speaker/embedding components and trained smaller discussion models.

2. Why this project? Recordings are difficult to search and decisions need context. We help a person locate relevant passages and inspect the original evidence.

3. Why the name InfluenceIQ? Influence reflects contribution to decision discussion, and IQ suggests intelligent assistance. We use it as branding, not a validated personal intelligence score.

4. Is this just a transcription app? Transcription is the foundation. The application adds speakers, slide associations, structured candidates, source links, reviewed follow-ups, contribution features, recovery, and exports.

5. What is novel? Our concrete source-preserving integration and exploratory feature comparison are the supported contributions. Established algorithms are reused. Integration alone does not prove algorithmic novelty or superiority over every meeting assistant.

6. Why not just use a commercial meeting assistant? Our implemented focus is local, inspectable review with explicit component boundaries. We have not performed a matched commercial-system benchmark, so we do not claim every commercial alternative is worse or lacks these features.

7. Where is the AI if decisions use rules? ASR, speaker embeddings, sentence embeddings, and supervised discussion classifiers are learned components. Rules connect their outputs into transparent candidate structures.

8. Are you using generative AI? Whisper generates recognized token sequences from audio. The deployed product does not use an LLM to generate meeting notes or chat answers. Optional text-model research is separate.

Audio, speakers, and performance

9. Why 16 kHz mono? It supplies compatible audio to the configured speech/speaker models and standardizes differing recording formats. It does not recover missing acoustic information.

10. What does int8 mean? Quantized inference uses lower-precision representations to reduce compute/storage pressure. It is an inference choice, not a statement that we trained a new model.

11. Why Whisper small? It is the configured CPU baseline. Larger alternatives were measured separately and did not improve every cohort/metric. Model choice also affects resource use.

12. Did you train Whisper or MiniLM? No. The deployed speech and sentence models are pretrained/frozen. We trained the small discussion classifiers and separate research rankers.

13. Is faster-whisper a different AI task? No. It is the local inference implementation around Whisper weights, using CTranslate2 execution.

14. What is VAD? Voice activity detection finds likely speech. The full orchestrator uses faster-whisper VAD filtering; the standalone Silero module serves a separate stage path.

15. What is a speaker embedding? A learned numerical representation useful for comparing voice characteristics. Clustering similar representations supports anonymous speaker groups.

16. How do you know the real speaker's name? Diarization does not establish a real identity. Users can review names and explicit self-introduction suggestions; the underlying cluster assignment remains a separate issue.

17. What if two people speak together? Temporal attribution can be ambiguous and return UNKNOWN. We do not deploy acoustic source separation that perfectly extracts overlapping voices.

18. Why use whole-meeting speaker clustering? Identity should remain consistent across the meeting. Independent chunk clustering would need another cross-chunk association mechanism. The current speaker stage therefore has a full-waveform memory cost.

19. Why chunk recognition at 120 seconds? It supports checkpoints and long recordings with surrounding context. The value is an implementation tradeoff, not a universally optimal duration.

20. Why two seconds of overlap? It supplies boundary context. Word-midpoint ownership avoids intentionally keeping a word in both central chunks.

21. Do you transcribe twice? Draft text and final attribution are separate passes, but unchanged word observations are cached. Speaker correction can reuse recognition rather than decode all words again.

22. Why was processing slower? Inspect stage timings and settings. In today's 90-second run, ASR took 17.36 seconds and speakers 23.83 seconds. Comparing against a transcript-only or cached run changes the workload. Larger saved model choices also matter.

23. Is it real time? Browser capture has bounded draft previews, and full processing exposes intermediate chunks. Final global speaker and decision analysis is offline. “Real-time final meeting understanding” would overstate it.

24. What if the computer stops? Retained audio, durable job state, and completed chunks support recovery. The current unfinished chunk may need repeating. Atomic writes help avoid partially written JSON/final audio files.

Modalities and fusion

25. Which modalities are used? Audio and optional presentation documents. Their alignment operates largely on transcript text and extracted slide text, with optional timing.

26. Are timestamps, speaker labels, and transcript three modalities? They are different derived signals from one recording. We do not count them as three independently captured modalities.

27. What fusion technique did you implement? Weighted score-level, or late, fusion of temporal overlap and semantic similarity. The joined source record propagates those links into later analysis.

28. Why late fusion? The existing modular pipeline produces separate interpretable scores and handles missing timing explicitly. We have not established that it is superior to all earlier/joint fusion alternatives.

29. Did you implement early fusion or cross-attention? No deployed joint acoustic-image encoder or cross-attention network exists. The discussion classifier concatenates lexical/context feature matrices, which is feature combination within a text task, not acoustic-image early fusion.

30. What is cosine similarity? A normalized dot product measuring vector direction similarity. In this project it indicates text relevance, not factual entailment or a calibrated correctness probability.

31. What happens without slide timestamps? Timing remains unavailable and the default content score receives the effective weight. Slide order alone does not provide display times.

32. What if all slides are unrelated? The best match must reach the configured threshold. Otherwise the output is NO_CONFIDENT_SLIDE. Even accepted links require review.

33. Does TF-IDF run together with MiniLM? They are alternative content scorers per run. Available timing can combine with the selected scorer. The current default uses MiniLM embeddings.

34. Can the system understand a chart image? Native text extraction and optional OCR read text. No general visual reasoning over diagrams/charts is established. OCR is disabled in the current settings.

35. Did hybrid fusion beat semantic-only? Not on the constructed 24-case diagnostic: semantic-only was 83.33% top-1 and hybrid 66.67%. This reveals limitations of fixed timing weights. In the real untimed deck, hybrid and semantic-only coincide, so no temporal benefit can be claimed there.

36. Why not just use the current slide? A speaker can refer to an earlier/offscreen slide or say something unrelated. Temporal context alone can make the wrong link.

Decisions and contribution

37. What is an event? A source-linked candidate role for a passage, such as proposal, question, evidence, revision, or action item. The Python default selects one winning type and can abstain.

38. Does an agreement prove a decision? No. It can support a probable candidate under the rules. Agreement may mean acknowledgment rather than formal adoption.

39. Why are decision labels called candidates? An explicit cue may be misrecognized or misclassified, and the cluster is inferred. “Confirmed” is a system status based on detected event type, not independent adjudication.

40. How do you connect events? Type constraints and time windows provide evidence relations, candidate clusters, and interaction edges, while preserving endpoint IDs and recording times.

41. Does an arrow mean causation? No. For example, a challenge followed by revision is a temporal association. It does not establish that the challenge caused the revision.

42. What is influence in the app? A weighted sum of seven meeting-normalized contribution features. It is an estimate of observed decision-linked activity, not personal ability or causal impact.

43. Why not just count speaking time? Speaking time measures participation. Contribution can involve a short important objection or evidence statement. We still retain simple participation baselines because they can be competitive.

44. Were the seven weights learned? The deployed weights are prototype configuration values. Weight/ranker training tools exist separately; their outputs are not automatically installed as app defaults.

45. Why normalize by the meeting maximum? It puts differently sized feature counts on a common within-meeting scale. It makes the scores relative to that meeting and its participants.

46. Why does the highest score sometimes equal 0.7? Not every category is active. If active weights total 0.7, maximizing all those categories can give 0.7. The final score is not forced to make a winner equal one.

47. Why are many scores zero? No scored decision-linked feature was extracted for those groups. Missed cues, incomplete relationships, and speaker fragmentation can cause that; it does not establish no human influence.

48. What did you fix in that stage? A single recognized English proposal/evidence cue could score below the default cutoff solely because its bank contained many alternatives. Confidence now credits observed cues, and new reports expose scoring version and coverage. The new behavior needs independent accuracy evaluation.

49. Are these scores PageRank? No. Interactions can be represented as a graph, but the deployed influence score is a maximum-normalized weighted feature sum.

50. Do the scores sum to 100 percent? No. Each feature is normalized independently across people and then weighted. This is not a percentage allocation or a softmax probability distribution.

Learning and evaluation

51. What exactly did your discussion model learn? Whether a reference dialogue act belongs to an annotated decision discussion. It did not learn final-decision adoption or causal participant influence.

52. Why use word and character features? Word features capture terms/phrases; character n-grams capture useful subword forms. Their benefit must be assessed under matched comparisons rather than assumed.

53. Why ±40 context and ±5 smoothing? Those are the installed development-selected settings. Both count input units, not seconds. Future context is suitable for offline review but prevents a strict causal-streaming interpretation.

54. Why combine random forest and logistic regression? That specific 75/25 combination was selected in the development experiments. We do not claim mixtures are inherently better in every setting.

55. What does 53.54% mean? Historical F1 for English discussion membership on human reference acts. Precision was 39.94%, recall 81.18%. It is not complete app or final-decision accuracy.

56. Why is the always-negative accuracy higher? Most examples are negative. Predicting negative everywhere reaches 77.72% accuracy while retrieving zero positive discussions. F1/recall and other metrics reveal that failure.

57. What does 68.93% mean? Meeting-average pairwise participant-ordering accuracy against GAP's outcome-alignment proxy in the latest controlled exploratory study. It is a separate task from discussion detection.

58. What is GAP AII? Distance between an individual's pre-discussion ranking and the group's final ranking. Someone can begin close to the outcome without persuading anyone, so AII is not a direct causal label.

59. How did you prevent leakage? Keep complete project/meeting groups separated, fit transforms inside fitting partitions, and select parameters on development/inner folds. Prior researcher exposure still makes current historical analyses exploratory.

60. What is an ablation? Remove an input/feature under a controlled protocol and measure the effect. The latest Ridge comparison refits after removing participation or contribution features. Deleting fixed-score weights is a different diagnostic.

61. What are WER and DER? WER is word substitutions plus deletions plus insertions divided by reference words. DER measures missed speech, false alarms, and speaker-confusion time under a specified protocol. Lower is better; they measure different errors.

62. Does 100 minus WER equal accuracy? Not generally. Insertions can push WER above 100%, and it is not a complete understanding measure.

63. Does 656 passing tests mean high AI accuracy? No. It establishes tested software behavior. Accuracy requires task-appropriate independent references, such as checked words, speaker intervals, or contribution judgments.

64. Why are some metrics null or unavailable? The necessary ground truth is absent. Coverage/agreement and generated influence scores cannot substitute for independent reference accuracy.

65. What did the AI-labeled pilot prove? It measured agreement with one assistant's labels on a tiny curated cohort, not independent human validity. The selected paired model did not beat the majority baseline.

Interface, deployment, and next work

66. Why React and FastAPI? React manages the interactive capture/review state; FastAPI validates requests and connects directly to Python processing. These are practical project choices, not evidence that other frameworks cannot work.

67. Why SQLite instead of a database server? The current local/small-team design benefits from simple durable transactions. Distributed writable replicas would need a different storage/job architecture.

68. Why one background worker? It avoids overlapping heavy model workloads and simplifies persistent state on this machine. It limits throughput; this is not a high-volume multi-tenant service.

69. Can renaming a person fix diarization? No. It changes labels. Speaker correction requires rerunning acoustic grouping/attribution and reviewing the resulting clusters.

70. Are owners and dates automatically understood? Task candidates are extracted; humans confirm owners, exact dates, and state. The source text/time is preserved and stale reviews are not silently applied to changed evidence.

71. Is calendar export a notification service? No. It writes an ICS file for reviewed open tasks with explicit dates. The user chooses where to import it; later edits do not automatically update an external calendar.

72. Is the browser doing the AI inference? The browser captures audio and displays results. The Python backend runs the laptop/server models. Native Android can run its separate models on-device.

73. What does offline mean after deployment? The inference machine can use provisioned local models without contacting a model provider. A remote browser still needs a connection to upload to that machine. Native Android has an independent offline path.

74. Is local processing the same as encrypted storage? No. There are account/ownership controls, but local artifacts are not an encrypted vault. We have not certified a hardened public service.

75. Can you deploy only the React frontend? It can be served statically, but analysis still needs the Python API, FFmpeg, local model files, and persistent storage. A frontend URL alone is not the complete deployment.

76. What is ready for deployment? The local built app, model provisioning, one-worker launcher, environment examples, and reverse-proxy template exist. Public hosting, domain, and persistent server configuration still need an actual target and validation.

77. Does a GPU solve all runtime problems? A compatible GPU can accelerate configured ASR; it does not automatically accelerate the CPU speaker stage, uploads, or every other component. This session's reported run was CPU-based.

78. What would you improve next? Independently validate event/outcome links and speakers on fresh recordings, measure matched with/without-slide conditions, calibrate thresholds using development data, and evaluate resource use and physical-device behavior. Additional models are hypotheses until the right evidence supports them.

Project ownership and data

79. What was your personal contribution? State the work you actually performed, then show its files, inputs/outputs, tests, and a design decision you can explain. The suggested slide handoff is not a record of individual implementation authorship. Do not invent ownership because a name appears beside a presentation block.

80. Which datasets did you use and why? AMI for meeting speech and manually referenced discussion; GAP for outcome-alignment ranking; eight FLEURS Hindi and eight MUCS-Hinglish samples for small recognition pilots. Each dataset serves a different task. Constructed slide fixtures test specific alignment behavior without establishing real-meeting multimodal accuracy.

9. Deployment position for this presentation

Local deployment is already working at port **8001**. Setup checks confirm local ASR, speaker, slide, and OCR assets; OCR is configured off. Shell syntax and launcher help were checked. This is readiness evidence, not a public deployment.

Use this wording: “The complete application is deployed locally on our workstation. For remote use, we will run the Python worker and model assets on a persistent host, serve the built frontend through the backend/reverse proxy, and preserve database, recordings, reports, and secrets. Clients then connect to that host.”

The repository has `deployment/run_workstation.sh`, `deployment/workstation.env.example`, and `deployment/Caddyfile.example`. The sample launcher/proxy uses port 8000; today's running instance uses 8001. Do not start a second backend against the same runtime or assume the sample hostname is a real deployment.

A public target has not been specified or configured. No domain, cloud account, public endpoint, GPU provisioning, or remote data transfer was created for this presentation preparation. The next deployment step is target-specific; it should preserve the current working local demo.

10. Where to look when you need evidence

Topic	Authoritative local source
HTTP and accounts	`backend/api.py`
Complete pipeline	`src/pipeline/orchestrator.py`
ASR and attribution	`src/audio/asr.py`
Speaker backend	`src/audio/diarization.py`, `onnx_diarization.py`
Slide extraction / OCR	`src/ppt/extract.py`, `ocr.py`
Fusion	`src/fusion/multimodal_representation.py`, temporal/semantic/lexical modules
Events / relationships / decisions	`src/meeting/events.py`, `evidence.py`, `decisions.py`, `interaction.py`
Contribution scoring	`src/meeting/influence.py`
Learned discussion review	`src/meeting/discussions.py`, `models/discussion/manifest.json`
Follow-ups / exports	`src/meeting/followups.py`, `src/report/transcript_export.py`
Frontend	`frontend/src/App.jsx`, `ProcessingPanel.jsx`, `TaskReview.jsx`, `recordingStore.js`, `livePreview.js`
Current correction details	`docs/validation/30_WEBSITE_REGRESSION_2026_10_25.md`
Research scope	`docs/paper_20261024/manuscript.md` and `analysis/`

Final sentence to remember: **“Our system makes candidate interpretations inspectable; independent references determine whether those interpretations are correct.”**